CM2005 Report on Technical Analysis Toolkit Implementation

Introduction

The submitted coursework implements a technical analysis toolkit for visualizing and predicting weather data. A CSV reader (csvReader) was utilized to parse the dataset, enabling efficient extraction and processing of temperature data. The dataset spans from 1980 to 2019 and contains hourly temperature readings for various European countries.

This report provides a detailed breakdown of each task's implementation, logic, and fulfillment of the requirements, with code snippets and results sections included for illustration.

Task 1: Compute Candlestick Data

Implementation Overview

The goal of this task was to compute candlestick data for a specified time frame (e.g., yearly temperature for a country). The computation involves determining the Open, High, Low, and Close values for the temperature data.

Logic and Methods

1. **Candlestick Class Definition**: A Candlestick class was implemented with attributes:
    - date (time frame identifier)
    - open (average mean temperature of the previous frame)
    - high (highest temperature of the frame)
    - low (lowest temperature of the frame)
    - close (average mean temperature of the current frame)

```
class Candlestick {
public:
    double openTemp, closeTemp, highTemp, lowTemp;
    int year;

    Candlestick(int yr, double open, double close, double high, double low)
        : year(yr), openTemp(open), closeTemp(close), highTemp(high), lowTemp(low) {}
};
```

Computation Logic:

1. Initialize CSV Reader and Read Data

- The CSVReader class is used to read the weather data file.
- Data rows and headers are extracted, and a headerIndexMap is created to map column names (e.g., GB_temperature) to their indices.

```
    CSVReader csvReader(csvFilePath);
    auto rows = csvReader.readAll();
    auto headers = csvReader.getHeaders();

    std::vector<Candlestick> candlestickData;
    if (rows.empty() || headers.empty()) {
        std::cerr << "Error: Missing data or headers in the file.\n";
        return candlestickData;
    }

    std::map<std::string, int> headerIndexMap;
    for (size_t i = 0; i < headers.size(); ++i) {
        headerIndexMap[headers[i]] = i;
    }

    if (headerIndexMap.find(country) == headerIndexMap.end()) {
        std::cerr << "Error: Country '" << country << "' not found in the dataset.\n";
        return candlestickData;
    }
```

2. Verify Country Availability

- The country code (e.g., GB_temperature) is checked against the headerIndexMap.
- If the country is not found, an error message is displayed, and the function exits.

```
    if (headerIndexMap.find(country) == headerIndexMap.end()) {
        std::cerr << "Error: Country '" << country << "' not found in the dataset.\n";
        return candlestickData;
    }

    int countryIndex = headerIndexMap[country];
```

3. Initialize Tracking Variables

- Variables to track the current year (currentYear), highest (highestTemp), and lowest (lowestTemp) temperatures are initialized.
- previousYearTemperatures and currentYearTemperatures are vectors for storing temperature readings.

```
    int currentYear = 0;
    double highestTemp = std::numeric_limits<double>::lowest();
    double lowestTemp = std::numeric_limits<double>::max();
    std::vector<double> previousYearTemperatures, currentYearTemperatures;
```

4. Iterate Through Data Rows

- Each row represents an hourly temperature reading.
- The year is extracted from the first column, and the temperature for the specified country is read from the relevant column.

```
    for (size_t rowIdx = 1; rowIdx < rows.size(); ++rowIdx) {
        const auto& row = rows[rowIdx];
        if (row.size() <= countryIndex) continue;

        int year = std::stoi(row[0].substr(0, 4));
        double temp = std::stod(row[countryIndex]);
```

5. Switch to a New Year (Year Transition)

- If the year changes:
    - Compute Open temperature:
        - Average of previousYearTemperatures.
        - If previousYearTemperatures is empty (e.g., first year), set Open to NaN.
    - Compute Close temperature:
        - Average of currentYearTemperatures.
    - Store the candlestick data (year, Open, Close, High, Low) in candlestickData.
- Reset currentYearTemperatures, highestTemp, and lowestTemp for the new year.
- Update previousYearTemperatures to hold the completed year's data.

```
if (currentYear != year) {
    if (currentYear != 0 && !currentYearTemperatures.empty()) {
        double openTemp = previousYearTemperatures.empty() ? std::numeric_limits<double>::quiet_NaN()
                                                           : std::accumulate(previousYearTemperatures.begin(), previousYearTemperatures.end(), 0.0) / previousYearTemperatures.size()
        double closeTemp = std::accumulate(currentYearTemperatures.begin(), currentYearTemperatures.end(), 0.0) / currentYearTemperatures.size();

        candlestickData.emplace_back(currentYear, openTemp, closeTemp, highestTemp, lowestTemp);
    }

    previousYearTemperatures = currentYearTemperatures;
    currentYearTemperatures.clear();
    currentYear = year;
    highestTemp = std::numeric_limits<double>::lowest();
    lowestTemp = std::numeric_limits<double>::max();
}
```

6. Track Yearly Highs and Lows

- For each temperature reading in the current year:
    - Update highestTemp with the maximum temperature encountered so far.
    - Update lowestTemp with the minimum temperature encountered so far.

```
        currentYearTemperatures.push_back(temp);
        highestTemp = std::max(highestTemp, temp);
        lowestTemp = std::min(lowestTemp, temp);
```

7. Handle Final Year

- After processing all rows, compute the candlestick data for the last year in the dataset.

```
    if (!currentYearTemperatures.empty()) {
        double openTemp = previousYearTemperatures.empty() ? std::numeric_limits<double>::quiet_NaN()
                                                           : std::accumulate(previousYearTemperatures.begin(), previousYearTemperatures.end(), 0.0) / previousYearTemperatures.size();
        double closeTemp = std::accumulate(currentYearTemperatures.begin(), currentYearTemperatures.end(), 0.0) / currentYearTemperatures.size();

        candlestickData.emplace_back(currentYear, openTemp, closeTemp, highestTemp, lowestTemp);
    }
```

8. Return Candlestick Data

- Return a std::vector<Candlestick> containing candlestick data for each year.

```
    return candlestickData;
```

9. Display Candlestick Data (void)

This function is responsible for formatting and displaying the computed candlestick data in a tabular format for better readability in the console.

Key Features of the Function:

1. **Header Formatting:**
   - The first two lines create headers (Year, Open, Close, High, Low) and a separator line (56 '-' characters).
   - std::setw ensures each column has a fixed width for alignment.

```
void displayCandlestickData(const std::vector<Candlestick>& candlesticks) {
    std::cout << std::setw(8) << "Year" << std::setw(12) << "Open"
              << std::setw(12) << "Close" << std::setw(12) << "High"
              << std::setw(12) << "Low" << "\n";
    std::cout << std::string(56, '-') << "\n";
```

2. **Iterating Over Candlestick Data:**

- A for loop iterates through the std::vector<Candlestick> containing the candlestick data for each year.
- For each Candlestick object, its attributes (year, openTemp, closeTemp, highTemp, lowTemp) are printed in the respective columns.

3. **Numeric Formatting:**

- std::fixed and std::setprecision(2) ensure that temperatures are displayed with two decimal places.

```
                << std::setw(12) << std::fixed << std::setprecision(2) << candlestick.openTemp
```

10. Main

The main function serves as the entry point of the program. This is present in each task file. Its purpose is to:

1. **Collect User Input:**
   - Prompt the user to input the country code (e.g., GB_temperature).
   - Store the input in the country variable.
2. **Generate Candlestick Data:**
   - Call the generateCandlestickData function with the provided country code and CSV file path.
   - Handle cases where no data is available (e.g., invalid country code) by displaying an error message and exiting.
3. **Display Data:**
   - If candlestick data is successfully generated, the displayCandlestickData function is invoked to present the data in a tabular format.

Key main features:

Opening the CSV:

```
    std::string csvFilePath = "weather_data_EU_1980-2019_temp_only.csv";
    std::string country;
```

User Input:

```
    std::cout << "Enter Country Code (e.g., GB_temperature): ";
    std::cin >> country;
```

Generate Candlestick Data:

```
    auto candlestickData = generateCandlestickData(country, csvFilePath);
    if (candlestickData.empty()) {
        std::cerr << "No data available for the specified country.\n";
        return 1;
    }
```

Display Results:

```
    displayCandlestickData(candlestickData);
```

Final output for task 1:

```
PS C:\Users\user\Desktop\oop1> g++ csvReader.cpp task1.cpp -o main
PS C:\Users\user\Desktop\oop1> ./main
Enter Country Code (e.g., GB_temperature): GB_temperature
    Year        Open       Close        High         Low
-------------------------------------------------------
    1980         nan        8.71       23.05       -3.61
    1981        8.71        8.63       23.58       -6.53
    1982        8.63        9.20       23.86       -7.18
    1983        9.20        9.23       26.22       -3.09
    1984        9.23        9.10       25.09       -2.99
    1985        9.10        8.25       23.39       -7.07
    1986        8.25        8.10       24.64       -5.95
    1987        8.10        8.42       24.45       -7.13
    1988        8.42        9.17       23.98       -2.13
    1989        9.17        9.92       26.49       -1.32
    1990        9.92       10.06       29.48       -1.76
    1991       10.06        9.08       24.43       -6.01
    1992        9.08        9.25       24.38       -2.96
    1993        9.25        8.83       22.62       -3.54
    1994        8.83        9.43       24.68       -3.27
    1995        9.43        9.67       27.63       -5.33
    1996        9.67        8.52       25.44       -3.43
    1997        8.52        9.84       26.19       -3.94
    1998        9.84        9.62       24.12       -1.96
    1999        9.62        9.90       25.59       -3.20
    2000        9.90        9.60       24.31       -1.59
    2001        9.60        9.24       24.69       -2.80
    2002        9.24        9.95       24.62       -3.34
    2003        9.95        9.84       28.15       -2.58
    2004        9.84        9.89       25.03       -2.26
    2005        9.89        9.95       25.19       -2.24
    2006        9.95       10.25       28.17       -2.12
    2007       10.25       10.05       23.29       -2.40
    2008       10.05        9.53       24.56       -2.04
    2009        9.53        9.63       25.56       -4.29
    2010        9.63        8.42       23.66       -4.61
    2011        8.42       10.07       23.90       -2.28
    2012       10.07        9.16       23.79       -5.12
    2013        9.16        8.96       25.89       -2.77
    2014        8.96       10.34       24.60       -1.03
    2015       10.34        9.67       26.59       -2.60
    2016        9.67        9.68       26.95       -1.40
    2017        9.68        9.99       25.55       -1.51
    2018        9.99       10.04       27.34       -4.52
    2019       10.04        9.95       29.11       -2.72
```

Summary for Task 1

Task 1 focuses on computing candlestick data for yearly temperature trends, including Open, Close, High, and Low values. The implementation parses temperature data from a CSV file and aggregates it by year for a specified country. The Open and Close represent average temperatures at the start and end of the year, while High and Low capture the extreme values within the year. This task establishes a robust foundation for subsequent analysis and visualization tasks by organizing temperature data into a clear, structured format.

Task 2 Breakdown: Creating a Text-Based Plot of Candlestick Data

Objective

The task requires creating a text-based plot of candlestick data. Each "candlestick" represents the temperature data for a specific period, including Open, Close, High, and Low values. The task is a visualization exercise to present this data in a clear and structured text format.

1. Parsing CSV Data and Preparing Candlesticks

This part extracts temperature data from the CSV file and aggregates it into candlestick data for a specific country.

Key Steps:

- Use the CSVReader class to read the file.
- Map the headers to column indices for easy lookup.
- Compute Open, Close, High, and Low temperatures for each year.

```cpp
std::vector<Candlestick> parseCSVToCandlesticks(const std::string& location, const std::string& filePath) {
    CSVReader reader(filePath);
    auto data = reader.readAll();
    auto headers = reader.getHeaders();

    std::vector<Candlestick> candlesticks;
    if (data.empty() || headers.empty()) {
        std::cerr << "Error: Missing data or headers in the file." << std::endl;
        return candlesticks;
    }

    std::map<std::string, int> columnMap;
    for (size_t i = 0; i < headers.size(); ++i) {
        columnMap[headers[i]] = i;
    }

    if (columnMap.find(location) == columnMap.end()) {
        std::cerr << "Error: Location '" << location << "' not found in the dataset." << std::endl;
        return candlesticks;
    }
```

2. Aggregating Yearly Temperature Data

This part processes the rows of the CSV to compute yearly candlestick values. It handles:

- Transitioning to a new year.
- Aggregating Open, Close, High, and Low values for each year.

```cpp
    int locationIndex = columnMap[location];
    int currentYear = 0;
    std::vector<double> yearTemps, prevYearTemps;
    double yearlyhighTemp = std::numeric_limits<double>::lowest();
    double yearlylowTemp = std::numeric_limits<double>::max();

    for (size_t i = 1; i < data.size(); ++i) {
        const auto& row = data[i];
        if (row.size() <= locationIndex) continue;

        int year = std::stoi(row[0].substr(0, 4));
        double temp = stringToDouble(row[locationIndex]);
        if (std::isnan(temp)) continue;
```

```cpp
    if (year != currentYear) {
        if (!yearTemps.empty()) {
            double openTemp = prevYearTemps.empty() ? std::numeric_limits<double>::quiet_NaN() : std::accumulate(prevYearTemps.begin(), prevYearTemps.end(), 0.0) / prevYearTemps.size()
            double closeTemp = std::accumulate(yearTemps.begin(), yearTemps.end(), 0.0) / yearTemps.size();
            candlesticks.emplace_back(currentYear, openTemp, closeTemp, yearlyhighTemp, yearlylowTemp);
        }
        prevYearTemps = yearTemps;
        yearTemps.clear();
        currentYear = year;
        yearlyhighTemp = std::numeric_limits<double>::lowest();
        yearlylowTemp = std::numeric_limits<double>::max();
    }

    yearTemps.push_back(temp);
    yearlyhighTemp = std::max(yearlyhighTemp, temp);
    yearlylowTemp = std::min(yearlylowTemp, temp);
}
```

3. Finalize Data for the Last Year

Ensures that the last year's candlestick data is also computed and stored.

```cpp
if (!yearTemps.empty()) {
    double openTemp = prevYearTemps.empty() ? std::numeric_limits<double>::quiet_NaN() : std::accumulate(prevYearTemps.begin(), prevYearTemps.end(), 0.0) / prevYearTemps.size();
    double closeTemp = std::accumulate(yearTemps.begin(), yearTemps.end(), 0.0) / yearTemps.size();
    candlesticks.emplace_back(currentYear, openTemp, closeTemp, yearlyhighTemp, yearlylowTemp);
}

return candlesticks;
```

Plotting the graph (void)

1.Mapping Temperatures to Plot Positions

This part translates the High, Low, Open, and Close temperatures to positions on a vertical scale for plotting.

```
void plotCandlestickGraph(const std::vector<Candlestick>& candlesticks) {
    const int graphHeight = 10;
    for (const auto& candle : candlesticks) {
        std::cout << "Year " << candle.year << ":\n";
        std::cout << "  openTemp: " << candle.openTemp << " | closeTemp: " << candle.closeTemp
                  << " | highTemp: " << candle.highTemp << " | lowTemp: " << candle.lowTemp << "\n";

        double range = candle.highTemp - candle.lowTemp;
        int openTempPos = static_cast<int>((candle.openTemp - candle.lowTemp) / range * (graphHeight - 1));
        int closeTempPos = static_cast<int>((candle.closeTemp - candle.lowTemp) / range * (graphHeight - 1));
        int highTempPos = 0;
        int lowTempPos = graphHeight - 1;
```

2.Handling Overlaps Between Open and Close

If Open and Close positions overlap, they are adjusted for better visualization.

```
        // Adjust for overlaps
        if (openTempPos == closeTempPos) {
            if (closeTempPos < graphHeight - 1) {
                ++closeTempPos; // Shift closeTemp marker one position up
            } else {
                --openTempPos; // Shift openTemp marker one position down
            }
        }
```

3.Creating and Printing the Temperature Scale

Builds a temperature scale and prints rows corresponding to each scale value, adding markers for High, Low, Open, and Close. High and low are represented by a T and B respectively.

```
        // Create temperature scale
        int minTemp = static_cast<int>(std::floor(candle.lowTemp / 5.0)) * 5;
        int maxTemp = static_cast<int>(std::ceil(candle.highTemp / 5.0)) * 5;
        std::vector<int> tempScale;
        for (int temp = maxTemp; temp >= minTemp; temp -= 5) {
            tempScale.push_back(temp);
        }

        std::map<int, int> tempRows;
        for (size_t i = 0; i < tempScale.size(); ++i) {
            int row = static_cast<int>((candle.highTemp - tempScale[i]) / range * (graphHeight - 1));
            tempRows[row] = tempScale[i];
        }

        for (int i = 0; i < graphHeight; ++i) {
            // Print temperature scale
            if (tempRows.find(i) != tempRows.end()) {
                std::cout << std::setw(4) << tempRows[i] << " ";
            } else {
                std::cout << "     ";
            }

            // Plot markers
            if (i == highTempPos) std::cout << " T  ";
            else if (i == lowTempPos) std::cout << " B  ";
            else if (i == openTempPos) std::cout << " O  ";
            else if (i == closeTempPos) std::cout << " C  ";
            else std::cout << " |  ";

            std::cout << '\n';
        }

        std::cout << '\n';
    }
}
```

Main

Handles user input for the country code and calls the functions to generate and plot candlestick data. Similar to Task 1.

```
int main() {
    std::string filePath = "weather_data_EU_1980-2019_temp_only.csv";
    std::string location;
    std::cout << "Enter the country code (e.g., GB_temperature): ";
    std::cin >> location;

    auto candlestickData = parseCSVToCandlesticks(location, filePath);
    if (candlestickData.empty()) {
        std::cerr << "No data available for the specified country." << std::endl;
        return 1;
    }

    plotCandlestickGraph(candlestickData);
    return 0;
}
```

Final Output for Task 2:

```
PS C:\Users\user\Desktop\oop1> g++ csvReader.cpp task2.cpp -o main
PS C:\Users\user\Desktop\oop1> ./main
Enter the country code (e.g., GB_temperature): GB_temperature
Year 1980:
  openTemp: nan | closeTemp: 8.70978 | highTemp: 23.054 | lowTemp: -3.612
  25  T
  20  |
  15  |
      |
  10  C
      |
   5  |
   0  |
      |
  -5  B

Year 1981:
  openTemp: 8.70978 | closeTemp: 8.63427 | highTemp: 23.579 | lowTemp: -6.533
  25  T
  20  |
  15  |
      |
  10  O
   5  C
      |
   0  |
  -5  |
      B
```

```
Year 2019:
  openTemp: 10.0382 | closeTemp: 9.94746 | highTemp: 29.105 | lowTemp: -2.719
  30  T
  25  |
  20  |
  15  O
      C
  10  |
   5  |
      |
   0  |
  -5  B
```

The output displays all results from 1980 to 2019, I have screenshotted only the start and end to avoid unnecessary image clutter

Summary for Task 2

Task 2 involves creating a text-based plot of candlestick data, visually representing yearly temperature trends with markers for Open, Close, High, and Low values. The implementation dynamically maps temperature ranges to a vertical scale and handles overlapping markers for clarity. This simple yet effective visualization provides users with an accessible way to analyse temperature variations over time without relying on graphical libraries, ensuring compatibility with a command-line interface.

Introduction for Task 3

In Task 3, the objective is to extend the candlestick visualization by implementing filtering options and generating a text-based graph. The filters allow users to refine data based on specific criteria, such as date range or temperature range. Additionally, users can reset filters to restore the original dataset. The filtered candlestick data can then be displayed in a structured format or plotted as a dynamic text-based graph.

In this section, I will demonstrate:

1. **Filtering Options**: How the program filters data by date range or temperature range, and resets filters.
2. **Displaying Candlestick Data**: A structured view of the candlestick attributes (Open, Close, High, and Low) for filtered results.
3. **Text-Based Graph**: A dynamically scaled graph representing the candlestick data, with markers for key values (Open, Close, High, and Low).

Each functionality will be explained with code snippets and examples to illustrate the implementation and its output.

1. Processing CSV Data

This part parses the CSV file, maps data to columns, and organizes temperature readings by year. This part is the same as task 1 and 2, therefore there is no need to go further in depth.

2. Computing Candlestick Data

Each year's temperature data is processed to compute Open, Close, High, and Low values. Similarly, this step has already been demonstrated in task 1 and 2, therefore I will not be going further in depth.

4.Filtering Options

Filter menu:

```
    if (candlestickData.empty()) {
        std::cerr << "No data available for the selected country." << std::endl;
        return 1;
    }

    while (true) {
        std::cout << "\nOptions:\n"
                  << "1. Display Data\n"
                  << "2. Plot Graph\n"
                  << "3. Reset Filters\n"
                  << "4. Filter by Date Range\n"
                  << "5. Filter by Temperature Range\n"
                  << "6. Exit\n"
                  << "Enter your choice: ";
        int choice;
        std::cin >> choice;

        static std::vector<Candlestick> filteredData = candlestickData;

```

Filtering allows users to dynamically refine the dataset based on specific criteria:

Filtering by Date Range

This filter keeps only candlesticks within the specified start and end years.

Logic:

- Use std::remove_if to iterate over the candlestick vector.
- Remove entries where the year falls outside the range [startYear, endYear].

```
        } else if (choice == 4) {
            int startYear, endYear;
            std::cout << "Enter start year: ";
            std::cin >> startYear;
            std::cout << "Enter end year: ";
            std::cin >> endYear;
            filteredData.erase(std::remove_if(filteredData.begin(), filteredData.end(),
                                              [startYear, endYear](const Candlestick& c) {
                                                  return c.year < startYear || c.year > endYear;
                                              }),
                               filteredData.end());
            std::cout << "Data filtered by date range." << std::endl;
```

Filtering by Temperature Range

This filter retains only candlesticks where both highTemp and lowTemp fall within the specified temperature range.

Logic:

- Use std::remove_if to remove candlesticks that do not meet the temperature criteria.

```
        } else if (choice == 5) {
            double minTemp, maxTemp;
            std::cout << "Enter minimum temperature: ";
            std::cin >> minTemp;
            std::cout << "Enter maximum temperature: ";
            std::cin >> maxTemp;
            filteredData.erase(std::remove_if(filteredData.begin(), filteredData.end(),
                                              [minTemp, maxTemp](const Candlestick& c) {
                                                  return c.highTemp > maxTemp || c.lowTemp < minTemp;
                                              }),
                               filteredData.end());
            std::cout << "Data filtered by temperature range." << std::endl;
```

Resetting Filters

This option restores the filteredData vector to its original unfiltered state.

Logic:

- Reassign filteredData to the unmodified candlestickData.

```
        } else if (choice == 3) {
            filteredData = candlestickData;
            std::cout << "Filters reset." << std::endl;
```

5. Displaying Candlestick Data

This option prints the filtered candlestick data in a structured format for inspection.

Logic:

- Iterate over the filtered dataset and display the attributes (year, openTemp, closeTemp, highTemp, lowTemp).
- Use std::setw for alignment and std::setprecision for formatting floating-point values.

```
if (choice == 1) {
    for (const auto& c : filteredData) {
        std::cout << "Year " << c.year << " | openTemp: " << c.openTemp
                  << " | closeTemp: " << c.closeTemp << " | highTemp: " << c.highTemp
                  << " | lowTemp: " << c.lowTemp << "\n";
    }
```

6. Plotting the Graph

The plotTextGraph function creates a text-based graph with markers for each year's candlestick data (High, Low, Open, Close).

Logic:

Determine Temperature Scale:

Calculate globalMin and globalMax across all candlesticks for scaling.

Round these to the nearest multiple of 5 for consistent tick intervals.

```
// Determine the global min and max temperatures for scaling
double globalMin = std::numeric_limits<double>::max();
double globalMax = std::numeric_limits<double>::lowest();

for (const auto& c : data) {
    globalMin = std::min(globalMin, c.lowTemp);
    globalMax = std::max(globalMax, c.highTemp);
}

// Adjust globalMin and globalMax to the nearest multiple of 5
int roundedMin = static_cast<int>(std::floor(globalMin / 5.0)) * 5;
int roundedMax = static_cast<int>(std::ceil(globalMax / 5.0)) * 5;
```

Map Temperatures to Rows:

- Scale Open, Close, High, and Low temperatures to row indices based on the graph height.
- Adjust for overlaps between markers using the helper function findAvailableRow.

```
        // Map each temperature to a row on the grid and place the corresponding marker
        for (const auto& marker : markers) {
            int row = static_cast<int>((marker.first - roundedMin) / (roundedMax - roundedMin) * plotHeight);
            row = findAvailableRow(row, i); // Adjust the row if the target row is occupied
            grid[row][i] = marker.second;
        }
    }
```

Render the Graph:

- Print temperature ticks on the left.
- Print markers for each year in columns corresponding to candlesticks.

```
    // Generate the graph
    for (int row = plotHeight; row >= 0; --row) {
        int currentValue = roundedMin + (row * tickInterval);

        std::cout << std::setw(6) << currentValue << " |";

        for (size_t col = 0; col < data.size(); ++col) {
            std::cout << std::setw(yearSpacing) << grid[row][col];
        }

        std::cout << "\n";
    }
```

Add Year Labels:

- Center year labels beneath the graph for easy identification.

```
    // Print year labels centered below the graph
    std::cout << std::setw(6) << " " << " ";
    for (const auto& c : data) {
        std::cout << std::setw(yearSpacing) << c.year;
    }
    std::cout << "\n";
}
```

Final Output for Task 3:

```
g++ csvReader.cpp task3.cpp -o main
PS C:\Users\user\Desktop\oop1> ./main
Enter the countrycode_temperature (e.g., GB_temperature): GB_temperature

Options:
1. Display Data
2. Plot Graph
3. Reset Filters
4. Filter by Date Range
5. Filter by Temperature Range
6. Exit
Enter your choice: 4
Enter start year: 2015
Enter end year: 2018
Data filtered by date range.

Options:
1. Display Data
2. Plot Graph
3. Reset Filters
4. Filter by Date Range
5. Filter by Temperature Range
6. Exit
Enter your choice: 2

Temperature Values:
Year 2015: openTemp: 10.3 | closeTemp: 9.7 | highTemp: 26.6 | lowTemp: -2.6
Year 2016: openTemp: 9.7 | closeTemp: 9.7 | highTemp: 26.9 | lowTemp: -1.4
Year 2017: openTemp: 9.7 | closeTemp: 10.0 | highTemp: 25.5 | lowTemp: -1.5
Year 2018: openTemp: 10.0 | closeTemp: 10.0 | highTemp: 27.3 | lowTemp: -4.5

    45 |
    40 |      T       T              T
    35 |                     T
    30 |
    25 |
    20 |
    15 |      O       C       C       C
    10 |      C       O       O       O
     5 |
     0 |              B
    -5 |      B               B       B
           2015    2016    2017    2018

Options:
1. Display Data
```

T and B represent the Open and close temperatures , the has the ability to filter. Once the data is displayed, the code loops to ask the user if they would like to filter with different options.

Summary for task 3:

Task 3 successfully extends the functionality of the candlestick visualization toolkit by adding interactive filtering options and enhanced data presentation. Users can dynamically refine data by date or temperature range, reset filters to restore the original dataset, and display filtered results in a tabular format. The text-based graph visually represents the candlestick data with dynamically scaled markers for Open, Close, High, and Low temperatures, ensuring clarity and usability. These features provide a flexible and interactive way to analyse temperature trends efficiently.

Task 4: Predicting Data and Plotting

This task involves predicting temperatures for a specific date and plotting the predicted values on a text-based graph. Below is a detailed breakdown of each part of the code with explanations.

1. Safe String-to-Double Conversion

To handle potential parsing errors during data reading, this function converts strings to double safely.

Logic:

- Return NaN if the string is empty or invalid.
- Use std::stod for valid conversions.

```
// Function to safely convert a string to double
double safeStringToDouble(const std::string& str) {
    try {
        if (str.empty()) throw std::invalid_argument("Empty string");
        return std::stod(str);
    } catch (...) {
        return std::numeric_limits<double>::quiet_NaN();
    }
}
```

2. Date Validation

This function ensures that the input date follows the YYYY-MM-DD format.

Logic:

- Use a regular expression to match valid date formats.
- Return true for valid dates and false otherwise.

```
// Function to validate the date format (YYYY-MM-DD)
bool isValidDateFormat(const std::string& date) {
    const std::regex dateRegex(R"(^\d{4}-(0[1-9]|1[0-2])-(0[1-9]|[12]\d|3[01])$)");
    return std::regex_match(date, dateRegex);
}
```

3. Convert Date to Day-of-Year (DOY)

This function converts a date (YYYY-MM-DD) into the corresponding day of the year.

Logic:

- Use a static array to represent days in each month.
- Account for leap years when calculating DOY.

```cpp
// Function to convert YYYY-MM-DD to Day-of-Year (DOY)
int dateToDayOfYear(const std::string& date) {
    int year = std::stoi(date.substr(0, 4));
    int month = std::stoi(date.substr(5, 2));
    int day = std::stoi(date.substr(8, 2));

    static const int daysInMonth[] = { 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31 };
    int doy = day;

    for (int i = 0; i < month - 1; ++i) {
        doy += daysInMonth[i];
    }

    // Account for leap years
    if (month > 2 && (year % 4 == 0 && (year % 100 != 0 || year % 400 == 0))) {
        ++doy;
    }

    return doy;
}
```

4. Reading Daily Temperature Data

This function reads temperature data from the CSV file and organizes it by DOY for the specified country.

Logic:

- Parse the header row to find the column corresponding to the country.
- Group temperatures by DOY in a map for efficient access.

```cpp
// Function to read daily data for a specific country
std::map<int, std::vector<double>> readDailyDataByDOY(const std::string& filePath, const std::string& country) {
    std::ifstream file(filePath);
    if (!file.is_open()) {
        std::cerr << "Error: Unable to open file: " << filePath << std::endl;
        return {};
    }

    std::string line;
    std::map<int, std::vector<double>> dailyTempsByDOY;

    // Parse the header row to find the column for the specified country
    std::getline(file, line);
    std::stringstream headerStream(line);
    std::string columnName;
    int countryColumnIndex = -1;
    int currentIndex = 0;

    while (std::getline(headerStream, columnName, ',')) {
        if (columnName == country) {
            countryColumnIndex = currentIndex;
            break;
        }
        currentIndex++;
    }

    if (countryColumnIndex == -1) {
        std::cerr << "Error: Country not found in the dataset.\n";
        return {};
    }
```

```cpp
    // Process daily data for the selected country
    while (std::getline(file, line)) {
        std::stringstream lineStream(line);
        std::string date, tempStr;
        std::getline(lineStream, date, ',');

        int doy = dateToDayOfYear(date);
        for (int i = 0; i <= countryColumnIndex; ++i) {
            std::getline(lineStream, tempStr, ',');
        }

        double temperature = safeStringToDouble(tempStr);
        if (!std::isnan(temperature)) {
            dailyTempsByDOY[doy].push_back(temperature);
        }
    }

    return dailyTempsByDOY;
}
```

5. Calculating Historical Averages

This function computes the historical average temperature for each DOY.

Logic:

- Iterate over the map of daily temperatures.
- Use std::accumulate to calculate the average for each DOY.

```
// Function to calculate the historical average temperature for each DOY
std::map<int, double> calculateHistoricalAverages(const std::map<int, std::vector<double>>& dailyTempsByDOY) {
    std::map<int, double> historicalAverages;

    for (const auto& entry : dailyTempsByDOY) {
        int doy = entry.first;
        const auto& temps = entry.second;

        double avgTemp = std::accumulate(temps.begin(), temps.end(), 0.0) / temps.size();
        historicalAverages[doy] = avgTemp;
    }

    return historicalAverages;
}
```

6. Plotting Predicted Temperature

This function generates a text-based graph for the predicted temperature on the target date.

Logic:

- Scale the Y-axis to fit the range of temperatures.
- Highlight the predicted temperature with a marker (*).

```
// Function to plot the temperature graph for a specific DOY
void plotTemperatureForDOY(const std::string& targetDate, double predictedTemp) {
    const int plotHeight = 20;
    const double tickInterval = 0.1; // Scale for Y-axis

    double minTemp = std::floor(predictedTemp - 1.0);
    double maxTemp = std::ceil(predictedTemp + 1.0);

    for (double tempValue = maxTemp; tempValue >= minTemp; tempValue -= tickInterval) {
        std::cout << std::setw(6) << std::fixed << std::setprecision(1) << tempValue << " | ";

        if (std::fabs(tempValue - predictedTemp) < tickInterval / 2) {
            std::cout << "*"; // Highlight predicted temperature
        } else {
            std::cout << " ";
        }

        std::cout << "\n";
    }

    // Print X-axis (target date)
    std::cout << "       " << targetDate << "\n";
}
```

7. Main Function

The main function integrates all components:

1. Reads temperature data.
2. Calculates historical averages.
3. Accepts user input for a target date.
4. Predicts temperature for the target DOY.
5. Plots the predicted temperature.

```cpp
int main() {
    std::string filePath = "weather_data_EU_1980-2019_temp_only.csv";
    std::string country;

    std::cout << "Enter the countrycode_temperature (e.g., GB_temperature): ";
    std::cin >> country;

    // Read data and compute historical averages
    auto dailyTempsByDOY = readDailyDataByDOY(filePath, country);
    if (dailyTempsByDOY.empty()) {
        std::cerr << "No data available for the selected country.\n";
        return 1;
    }

    auto historicalAverages = calculateHistoricalAverages(dailyTempsByDOY);

    // Input target date
    std::string targetDate;
    std::cout << "Enter the target date for prediction (YYYY-MM-DD): ";
    std::cin >> targetDate;

    // Validate date format
    if (!isValidDateFormat(targetDate)) {
        std::cerr << "Error: Invalid date format. Please use YYYY-MM-DD.\n";
        return 1;
    }

    // Check if the date is before 2020-01-01
    if (targetDate < "2020-01-01") { // String comparison works because YYYY-MM-DD is lexicographically ordered
        std::cerr << "Error: Cannot predict data for dates before 2020-01-01.\n";
        return 1;
    }

    int targetDOY = dateToDayOfYear(targetDate);
```

```cpp
    // Predict temperature for the target DOY
    if (historicalAverages.find(targetDOY) != historicalAverages.end()) {
        double predictedTemp = historicalAverages[targetDOY];
        std::cout << "\nPredicted Temperature for " << targetDate << ": "
                  << std::fixed << std::setprecision(2) << predictedTemp << "°C\n";

        // Plot the temperature graph for the specific DOY
        plotTemperatureForDOY(targetDate, predictedTemp);
    } else {
        std::cerr << "No historical data available for the specified date.\n";
    }

    return 0;
}
```

Final Output for Task 4:

```
Enter the countrycode_temperature (e.g., GB_temperature): GB_temperature
Enter the target date for prediction (YYYY-MM-DD): 2420-06-09

Predicted Temperature for 2420-06-09: 22.66°C
  24.0 |
  23.9 |
  23.8 |
  23.7 |
  23.6 |
  23.5 |
  23.4 |
  23.3 |
  23.2 |
  23.1 |
  23.0 |
  22.9 |
  22.8 |
  22.7 | *
  22.6 |
  22.5 |
  22.4 |
  22.3 |
  22.2 |
  22.1 |
  22.0 |
  21.9 |
  21.8 |
  21.7 |
  21.6 |
  21.5 |
  21.4 |
  21.3 |
  21.2 |
  21.1 |
       2420-06-09
```

Summary for task 4:

Task 4 leverages historical temperature data to predict future temperatures for a specific date. Using the average temperature for the same day across previous years is a straightforward and effective method for prediction, especially if there is a strong seasonal pattern in the data. The predicted temperature is then visualized on a dynamically scaled text-based graph. This task demonstrates advanced features such as date validation, day-of-year calculations, and graphical plotting to provide a robust and user-friendly prediction tool.

Limitations of the Prediction Function

The prediction function calculates temperatures based on historical averages, assuming consistent temperature patterns over time. This method does not account for variability due to climate change, extreme weather events, or other anomalies. Additionally, it operates solely on day-of-year (DOY) averages, ignoring factors like geographic variations or real-time trends, which limits its predictive accuracy.

The software predicts the temperature for a single date at a time. If the user wants to predict for another day, they must re-prompt the software by re-entering the desired target date. This requires starting the process again, as the program does not currently support batch predictions or retaining state for multiple queries.